The enterprise AI stack - a cheatsheet

Everything you need to know to have an informed conversation about AI in any enterprise setting. No jargon for the sake of jargon. If a concept is simple, it stays simple.

The 4 levels (and one you should probably ignore)

These are not steps you climb in order. They're a menu. Each business problem picks its own level. Most companies need level 1. Some need level 4. Almost none need level 5.

#	Level	What it does	When to use it	Cost (India)	Time
1	**RAG**	Connects an LLM to your company's data. System finds relevant documents and feeds them to the model before it answers. No training. Pure wiring.	Employees need to search internal docs, policies, contracts or knowledge bases using natural language.	10-30L to build 2-5L/month to run	4-8 weeks
2	**Fine-tuning**	Changes how the model behaves - its tone, format, terminology. You feed it examples of how you want it to write. It learns your style.	AI output needs to sound like your brand, follow your format or use your domain language.	5-15L per cycle	2-4 weeks
3	**Distillation**	Big model (teacher) answers thousands of questions. Small model (student) learns from those answers. 80% quality at 10% cost.	Paying too much for API calls. Need AI to run faster, cheaper or offline.	10-25L	4-6 weeks
4	**Classical ML**	Traditional ML on structured data. Random forests, gradient boosting, logistic regression. Predicts outcomes from patterns in numbers.	Structured data (spreadsheets, logs, transactions) and want to predict churn, risk, demand or failure.	20-50L to build 5-10L/month	3-6 months
5	**Own model**	Train a base LLM from scratch. 99% of companies should ignore this entirely.	Only if your product IS the model. Otherwise, don't.	50-100Cr+	12-18 months

Services companies sell them level 5 ambition at 50Cr. The person who right-sizes the solution to the actual problem - that's who earns trust.

The agent layer - what sits on top

An agent is not a level. It's the orchestration layer that ties the levels together. You give it a goal in plain English. It breaks the goal into steps, picks the right level for each step, runs them in order and gives you one combined result.

Think of it like a project manager. You say "prepare me for tomorrow's meeting." The PM doesn't do the work himself. He looks at his team (RAG for search, fine-tuned model for writing, prediction engine for anticipation), assigns each task to the right person, collects the output and delivers one clean briefing.

The 5 agent concepts

These are the building blocks of any AI agent - from a laptop project to ChatGPT plugins to enterprise automation systems.

Concept	What it means	Industry term
Task decomposition	Breaking a vague goal into concrete steps. "Prepare for meeting" becomes: predict files, search documents, write briefing.	Same everywhere. LangChain, CrewAI and AutoGen all use this term.
State management	Tracking what's done and what's next. A simple checklist: pending, running, done or failed.	Also called agent memory or scratchpad.
Tool selection	For each step, picking the right function to call. Search for finding, write for generating, predict for anticipating.	Called tool calling or function calling. When OpenAI or Anthropic say this, same concept.
Error recovery	When a tool fails, the agent doesn't stop. It logs the failure, skips the step and continues.	Called fault tolerance or graceful degradation.
Planning loop	Each step receives results from all previous steps as context. Step 3 knows what step 1 found.	Called ReAct (reason + act) in research papers.

> *The agent's quality depends on two things: how smart the planner model is (plan quality) and how good the tools are (execution quality). Better models make better plans. Better data makes better tools. The framework stays the same.*

From agent to autonomous agent

A standard agent plans and executes. If a step produces garbage, it moves on without noticing. An autonomous agent adds reflection - after every step, it evaluates its own output. If the result is poor, it retries with a different approach. It catches its own mistakes.

The industry calls this the ReAct pattern (reason, act, observe, reason again). Four new concepts sit on top of the 5 agent concepts:

Evaluate - after each step, check the result using simple rules (is it empty? too short? contains an error?) plus one LLM question (is this relevant?). **Retry** - when evaluation fails, change the approach. Simplify, rephrase or make it more specific. **Reflect** - log every attempt so you can see the agent's reasoning. **Converge** - max retries prevent infinite loops. Accept the best result and be honest about it.

> *Reflection can't fix a broken tool. But it tells you the tool is broken - instead of silently passing garbage to the next step. That's the difference between an agent that fails silently and an agent you can trust.*

Key concepts - what each word actually means

Embeddings

A way to turn text into numbers so machines can compare meaning. "I'm hungry" and "I want food" look different as text but their embeddings are very close together. This is how RAG finds the right documents - it compares embeddings of your question against embeddings of every chunk in your database. Closest match wins.

Vector database

A database optimized for storing and searching embeddings. Regular databases search by exact match. Vector databases search by similarity (find chunks whose meaning is closest to this question). ChromaDB, Pinecone and Weaviate all do the same thing.

Chunking

Breaking large documents into smaller pieces before embedding them. You can't feed a 200-page PDF into an embedding model. So you split it into paragraphs (chunks), embed each separately and store them individually. When someone asks a question, the system retrieves the most relevant chunks.

Context window

The maximum amount of text an LLM can read at once. Think of it as the model's working memory. GPT-4 handles ~128K tokens (~100 pages). Mistral 7B handles ~8K tokens (~6 pages). If your chunks plus the question exceed the context window, the model can't read all of it.

Tokens

The unit LLMs think in. Not words - pieces of words. "Enterprise" is 2-3 tokens. "AI" is 1. Roughly: 1 token is about 3/4 of a word. 1,000 tokens is about 750 words. You pay per token. You hit limits in tokens. Everything is measured in tokens.

LoRA (low-rank adaptation)

A technique that makes fine-tuning possible on normal hardware. Instead of retraining all 7 billion parameters (needs massive GPUs), LoRA freezes 99.9% and only trains a tiny adapter layer (0.1%). Nearly as good as full fine-tuning but runs on a laptop.

Temperature

Controls how creative vs predictable the model is. Temperature 0 = always picks the most likely next word (factual, repetitive). Temperature 1 = more randomness (creative, sometimes wrong). For RAG and enterprise use, keep it low (0.1-0.3).

Inference

When the model actually runs and produces an answer. Training teaches the model. Inference uses what it learned. API costs are inference costs. Latency complaints are inference speed complaints. "Scheduled inference" means running the model automatically at a set time.

TF-IDF

Term frequency - inverse document frequency. Measures which words in a question actually matter. "Show me the budget" - "show" and "me" appear everywhere (low importance). "Budget" appears rarely (high importance). TF-IDF gives "budget" a high score.

Feature engineering

Turning raw messy data into clean numbers a ML model can learn from. Your query log has timestamps and text. Feature engineering turns "Monday 9:15 AM" into day_of_week=1, hour=9, is_morning=1. The better your features, the better your predictions. This is the real skill in ML - not the algorithm.

The strategic layer - what the bible doesn't cover

Step zero: data readiness

Before you pick any level, assess: is the data even in a state where AI can use it? This is where 60% of enterprise AI projects die - not in model selection, but in data cleanup. A product leader who walks in and says "before we discuss RAG or fine-tuning, show me where your data lives and in what format" saves the company 6 months of wasted effort.

Questions to ask: Where does the data live? How many systems? What format? Who owns it? Is there PII that needs masking? How current is it? If nobody can answer these in 30 seconds, the data is not ready.

How to measure if AI is working

Level	The question	How to answer it
RAG	Are we retrieving the right documents?	Have 50 employees rate answers weekly. Track % rated useful.
Fine-tuning	Does the AI sound like us now?	Blind test - employees can't tell AI from a colleague.
Distillation	Is the small model as good as the big one?	Run same 1,000 questions through both. Compare side by side.
ML predictions	Are the predictions actually right?	Of 10 flagged as risky, how many actually failed? That's precision.
Agent	Does the agent complete the goal end to end?	Give it 20 goals. Track completion rate and useful results.
Autonomous	Does it catch its own mistakes?	Compare output with and without reflection. Count quality catches.

Build vs buy vs partner

Approach	When it makes sense	When it doesn't
Build	AI is your core product or a key differentiator. You have engineering talent. You need full control over data, models and iteration speed.	You're building AI because it sounds impressive, not because the problem demands it.
Buy (SaaS)	Standard use case - customer support, document search, content generation. Someone already built a good product.	Your data is sensitive and can't leave your infrastructure. The vendor's model doesn't understand your domain.
Partner	You need custom AI but don't have the team. A services partner builds it, you own the output.	You plan to run AI at scale long-term. Knowledge walks out when the partner leaves.

Who to hire

The most common enterprise mistake: hiring the wrong role for the wrong level.

Role	What they do	Covers	India salary
Software engineer	Builds the RAG pipeline, agent layer and autonomous agent framework. APIs, data ingestion, vector DB wiring, reflection loops.	Level 1 (RAG) + agent + autonomous agent	12-35L/year
ML engineer	Runs fine-tuning jobs, manages training pipelines, handles model deployment. Cares about training loss, learning rate, GPU utilization.	Level 2 + 3 (fine-tuning, distillation)	20-50L/year
Data scientist	Builds classical ML models from structured data. Designs features, tests hypotheses, evaluates predictions. Cares about precision, recall and feature importance.	Level 4 (classical ML)	18-45L/year
AI researcher	Designs new architectures, runs pre-training, publishes papers. Don't hire unless you're building a model company.	Level 5 only	40L-1Cr+/year

Common mistake: companies hire data scientists when they need software engineers (for RAG) or hire AI researchers when they need ML engineers (for fine-tuning). Wrong hire = 6 months wasted.

Red flags - what to watch for

These are the signals that an AI project, vendor or internal team is headed for trouble.

"We need our own model."

No you don't. Unless your product IS the model, you need someone else's model plus your data. RAG + fine-tuning covers 95% of enterprise use cases.

"We'll start with a proof of concept."

POCs without clear success criteria are where budgets go to die. Define what "working" looks like, who will judge it and what happens if it works. If the last answer is vague, the POC is theater.

"The data is ready."

It almost never is. Where does it live? How many systems? What format? Who owns it? If nobody can answer in 30 seconds, the data is not ready.

"We need an AI strategy."

You need a business problem that AI can solve. "Our support team takes 48 hours to resolve tickets - can AI bring that to 4?" is a real question. "We need an AI strategy" is not.

"We need GPUs."

For RAG? No. For fine-tuning with LoRA? Your laptop works. For inference? API calls. Most enterprises are nowhere near needing their own GPU infrastructure.

"Let's hire a Chief AI Officer."

A title won't save you. If you don't have a clear business problem, clean data and engineering capacity, a CAIO will spend 6 months writing PowerPoint decks. Level 1 needs a software engineer. Not a CxO.

"We need an agent."

Most companies asking for agents haven't finished level 1. An agent orchestrates tools - if the tools don't work well, the agent just orchestrates garbage faster. Get RAG right first. Then fine-tune. Then add the agent layer.

Quick reference - one-liners for each level

Designed to be said out loud, not read from a slide.

RAG: "Don't retrain the model. Just show it the right documents at the right time."

Fine-tuning: "RAG gives it knowledge. Fine-tuning gives it personality."

Distillation: "Make the expensive model teach the cheap model. Then fire the expensive model."

Classical ML: "LLMs are great at language. They're terrible at predicting numbers. Use the right tool."

Agent: "An agent without good tools is a project manager with no team. Fix the tools first."

Autonomous agent: "The agent that tells you 'this result is garbage' is more valuable than the one that says 'done.'"

Own model: "If you're asking whether you need this, you don't."
